NPLify · P0 Technical Baseline

Architecture & Stack Note

New XP Technologies Limited

22 October 2026 · Confidential

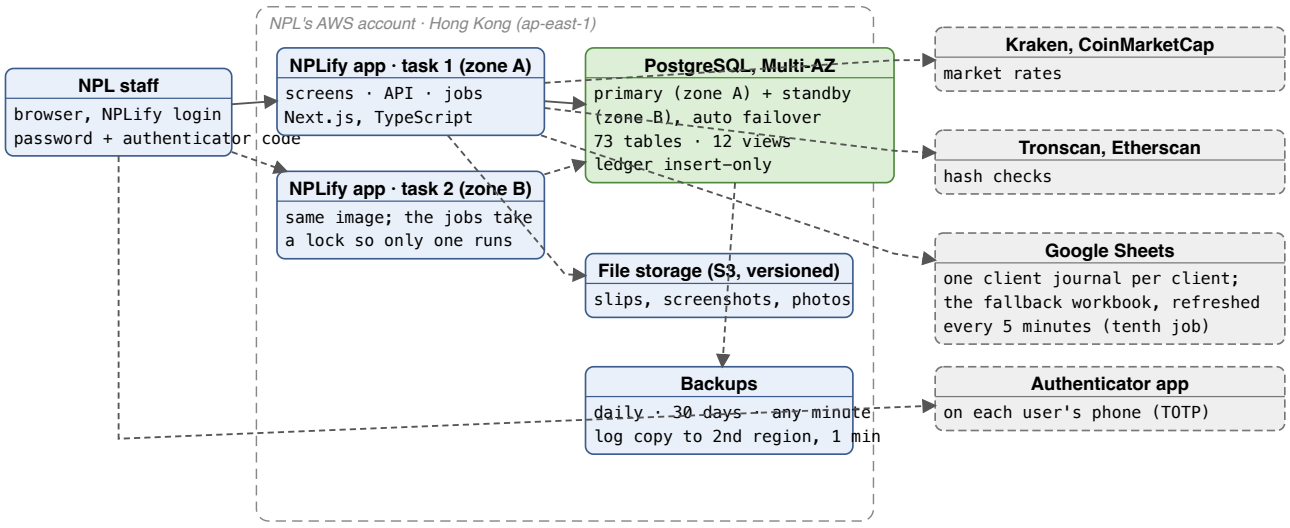
How to read this pack

Four map pages are now part of the pack as documents of their own, and are the place to start: the Top-Down Map (from the business, through the data model and the deal states, to the scenarios and the reading order), the Deal Flow on one page (Quote, Collect, Convert, Settle, with the roles and the money path), the Workflow Tree (8 classes, 27 sub-classes, 92 procedures, each naming who initiates, who approves, the codes it is written under and the records it touches) and the Workflow Tree on one page. Read them in that order, then this document. Every worked transaction (T), acceptance vector (L, B), walkthrough (W, X), flow (F) and procedure (C, A, D) has one identifier, listed in the Reading Companion's identifier register; the Test Strategy & Coverage document says which test proves each procedure.

1 · In one paragraph

NPLify is an internal tool. It runs as **one application** — screens, API and background jobs in a single Next.js / TypeScript service — against **one PostgreSQL database** whose ledger and audit tables accept inserts only, with **one S3 bucket** for slips, screenshots and photos, all inside **NPL's own AWS account in Hong Kong**. Staff sign in with NPLify's own login: email, password and a code from an authenticator app; users, roles and rights are rows in NPLify. Four outside services are used and none is trusted with the books: Kraken and CoinMarketCap for market rates, Tronscan and Etherscan to check transaction hashes, and one Google Sheet per client as that client's status journal. Daily backups are kept thirty days with restore to any minute; the backups and the transaction log are copied continuously to a second AWS region, the log every minute, and the evidence bucket is replicated there, and a live standby database runs in that region, so a whole-region failure is recovered within the same one hour. Running cost with the resilience decided on 20 October and the second-region copies of 28 October: about USD 205 to 300 a month (Section 6). At ten deals a weekday the whole model is a few thousand rows a month; nothing here is sized for more than that, and nothing needs to be.

Figure 1 — how it fits together, with the resilience decided on 20 October



Recovery (Cutover 8a): a zone failure fails over to the standby; a region failure promotes the second-region standby; one hour either way; the fallback workbook bridges the hours in between. Running cost about USD 205–300 a month (Section 6).

Figure 1 — Two application tasks in two zones, a Multi-AZ database, backups and the transaction log replicated continuously to a second region, and the fallback workbook as the tenth job. Recovery (Cutover Approach 8a): a zone failure fails over to the standby and the second task; a whole-region outage promotes the live standby database in the second region and starts the application there; one hour in both cases; the fallback workbook bridges the hours in between. Running cost about USD 205–300 a month (Section 6).

2 · Stack

Piece	NPL's page	What it has to do for the model (v6.0)
App	Next.js with TypeScript: screens, API and background jobs (quote clocks, rate fetch, alerts) in one app.	Screens per role (Section 5); a JSON API the screens call, which is also where every rule of the model is enforced — pricing (Calculation Specification), the state machines (Lifecycle & Approvals), maker-checker from ROLE_RIGHT rows, the field filter of ROLE.sees_economics; and a job runner inside the same service for the nine jobs of Section 4. One process, one deployment, one log.
Database	PostgreSQL. Ledger and audit tables accept inserts only; the database refuses updates and deletes.	75 tables and 12 derived views of the ERD. LEDGER_TRANSACTION, LEDGER_POSTING, LEDGER_ACCOUNT, AUDIT_LOG are insert-only at the database level (Section 3). Every balance anywhere is a SQL view over postings; nothing is stored twice. Money is `numeric`, never floating point; time is `timestamptz`, shown in SYSTEM_SETTING.operating_timezone and in each partner's cutoff_timezone where the partner's clock applies.

Piece	NPL's page	What it has to do for the model (v6.0)
Files	Amazon S3 for slips, screenshots and photos.	The model never stores a file in a table: it stores a reference. Objects go into one versioned bucket, private, encrypted at rest, reached through short-lived signed links the app issues; evidence files (bank letters, KYC, legal documents) sit under a restricted prefix only Finance and Management roles can resolve. Bank account details themselves are in the database (BANK_ACCOUNT_DETAILS, versioned), visible in full to Operations, Finance and Management, every full read logged in AUDIT_LOG. The bucket has versioning and object lock: an object is never overwritten or deleted inside its retention, its key carries the file's SHA-256 so a reference resolves to the bytes attached or to nothing (ERD Appendix C), and the bucket is replicated to the second region.
Sign-in	NPLify's own login: email and password plus a code from an authenticator app. Users and roles are managed in NPLify.	USER (email, role_id, status) → ROLE (sees_economics) → ROLE_RIGHT (may_initiate, may_approve, escalates_to_role_id). Password hashed with a modern algorithm; TOTP secret per user; a session cookie, server-side, that expires on inactivity; every user change approved by a second Management user (Roles & Visibility rule 7). No outside identity provider: the ERD's note "Google Workspace" on USER.email describes the address, not the login.
Hosting	AWS in Hong Kong: one small container service, a managed PostgreSQL database and an S3 bucket.	Region ap-east-1. Two small application tasks in two availability zones (App Runner or ECS on Fargate), and a Multi-AZ managed PostgreSQL instance with a standby in the second zone (Section 7, item 5, as decided on 20 October): RDS, smallest burstable class, storage auto-growing, automated backups on; one S3 bucket with versioning. Secrets (database password, rate-feed and explorer keys, the Sheets service account) in the account's secrets store, never in the image or the repository. Outbound calls only to the four services of Figure 1.

3 · The insert-only ledger, enforced by the database

NPL's page says the database refuses updates and deletes on ledger and audit tables. In the model, ledger tables are append-only and corrections are reversing transactions, and that is enforced in three layers so that no screen, no job and no mistake can get around it:

1. **Privileges.** The application's database role has INSERT and SELECT on `LEDGER_TRANSACTION`, `LEDGER_POSTING`, `LEDGER_ACCOUNT` and `AUDIT_LOG` and *no* UPDATE or DELETE. Nobody uses the owner role day to day; schema changes are migrations run by deployment under a separate role.
2. **Triggers.** A BEFORE UPDATE OR DELETE trigger on the same tables raises an error whatever the role, so even an operator connected as the owner cannot change a posting by hand. `LEDGER_ACCOUNT` allows one exception: a row's `status` may be changed to closed; its code, owner, holder, currency and purpose may not.

3. **Balance at commit.** A deferred constraint trigger on `LEDGER_POSTING` checks, when a transaction commits, that its postings sum to zero per currency and, separately, per ownership tag, that every `CL.PROJECT.<CCY>.PAYABLE` line carries a `receiver_group_id`, and that the transaction names a source record of the stated `source_type`. An unbalanced transaction never reaches the books.

Everything else in the model is a normal row with a lifecycle (a deal, a settlement, a conversion, an approval) or a versioned configuration row (a fee structure, a threshold, a template, a system setting: new rows with an effective date, never edits). For those, `AUDIT_LOG` holds every field change with old and new value and the acting user, written by the API in the same database transaction as the change, and `RECORD_LOCK` prevents two people editing one record at once. Reversal is the only correction: `LEDGER_TRANSACTION.reversal_of_id` names the transaction reversed, and the Adjustment source is a Finance-initiated, Management-approved action (Lifecycle & Approvals §5).

4 · Background jobs the model needs

NPL's page names quote clocks, rate fetch and alerts. The model's clocks and checks need the nine jobs below; all run inside the single app process on a schedule table, each run recorded with its outcome so the exception list can show a job that failed. None of them moves money: a job may stamp a clock, expire a quote, attach evidence or raise an alert, but every posting is made by a controlled action of a person (or by the engine as part of one).

Job	When	What it does	Tables
Rate capture	every 60 s per active pair	Walks each CURRENCY_PAIR's PAIR_RATE_SOURCE chain (project pairs and platform pairs alike): Kraken first, CoinMarketCap next; writes a MARKET_RATE row per source that answered; marks a source stale when its snapshot is older than RATE_SOURCE.staleness_threshold_s. With every source down the job raises an alert and pricing waits for a manual rate entered by Finance with evidence and approved; it is recorded like any other market rate, with its approval.	`MARKET_RATE`, `RATE_SOURCE`, `PAIR_RATE_SOURCE`, `CURRENCY_PAIR`
Quote clocks	every 30 s	Moves quote-first legs along the clocks of the project: freshness (re-price before sending), validity (effective_valid_until, extended by QUOTE_VALIDITY_EXTENSION rows), grace (grace_until with the drift test); expires what passed; a late sender gets a new deal group.	`DEAL`, `DEAL_GROUP`, `QUOTE_PACKAGE`, `QUOTE_VALIDITY_EXTENSION`, `PROJECT`
Partner clocks and business days	every minute, in each partner's cutoff_timezone	Applies the effective cutoff (one-day override, else weekday rule, else PARTNER.bank_cutoff_time) to legs on pairs with needs_bank_lock: stamps cutoff_at and conversion_due_date, defers a next-day conversion that would breach the minimum margin, moves a settlement approved after cutoff to the next business day (execution_due_date) skipping weekends and HOLIDAY rows. Expires PARTNER_RATE_VERSION rows at expires_at and freezes a breached partner × pair until Finance decides.	`PARTNER`, `PARTNER_CUTOFF_OVERRIDE`, `HOLIDAY`, `PARTNER_PAIR`, `DEAL`, `SETTLEMENT`, `PARTNER_RATE_VERSION`, `RATE_COMPARISON`
Hash checks	on demand and every 5 min for pending ones	For a crypto collection or payout whose hash was shared: asks Tronscan (TRC-20) or Etherscan (ERC-20) for the transaction, compares address, amount and confirmations, and attaches the result as evidence. If the explorer does not answer, the item stays pending and Operations verifies by the explorer link; the job never auto-verifies a collection — verification is an Operations action with the evidence attached.	`COLLECTION`, `SETTLEMENT_LINE`, `CONFIRMATION`, `COLLECTION_RECEIVING_ENDPOINT`, `RECEIVING_ENTITY_ACCOUNT`

Job	When	What it does	Tables
Aging and exceptions	every 15 min	Builds the exception list every role sees in its own terms (Roles & Visibility §5, rule 8): custody beyond OWN_WALLET.custody_max_hours, open OFFSET rows past their aging, a pool below zero outside the allowed window, unfunded make-goods, open loss recoveries, frozen pairs, variance exceptions, destination approvals pending, settlements due today, single-use endpoints past their deal. Sends alerts to the role that must act.	`CUSTODY, OFFSET, PROJECT_BALANCE, LOSS_EVENT, LOSS_RECOVERY, DESTINATION_APPROVAL, SETTLEMENT, APPROVAL`
Exposure and reporting values	at request time, cached 60 s	REPORTING_VALUE is computed when a screen or report asks for it — never stored: the latest MARKET_RATE of the platform pair, refused as stale beyond reporting_rate_max_age_min. The same job compares Σ balances at partners with PROJECT.exposure_ceiling and CLIENT.exposure_ceiling and blocks a release that would cross either.	`REPORTING_VALUE, SYSTEM_SETTING, CLIENT_POSITION, PROJECT_BALANCE`
Monthly accruals and statements	first business day of the month, and on demand	Rolls PROJECT_MONTH_VOLUME for tier selection; produces the partner rebate statement to reconcile PARTNER_REBATE_ACCRUAL against the partner's statement and the markup-share statement per party; lists EARNINGS_RECEIVABLE rows resting at each partner for Finance to settle or offset.	`PROJECT_MONTH_VOLUME, PARTNER_REBATE_ACCRUAL, MARKUP_SHARE_ACCRUAL, EARNINGS_RECEIVABLE, EARNINGS_OFFSET`
Client journal	on every deal-state change, retried until confirmed	Appends one row to the client's Google Sheet for each state change of a deal group (quoted, sent, collected, converted, settled, confirmed, returned, voided) with sender-facing fields only (Roles & Visibility rule 2), then reads the row back and records the confirmation. A failed write stays queued and is shown on the exception list; the deal does not wait for the sheet.	`DEAL_GROUP, DEAL, SETTLEMENT_LINE, MESSAGE_TEMPLATE`
Backup verification	daily, after the backup window	Confirms the day's automated snapshot exists and restores last week's snapshot into a scratch database once a week to prove the restore path; checks S3 versioning is on. Reports to Management monthly.	`all tables`

How the jobs stay reliable

Six rules apply to every job above, and to the tenth job added on 20 October, the fallback workbook export (every five minutes: open deals and their states, partner balances, the day's confirmed rates, cutoffs and pending approvals, written as two Google workbooks: an operational workbook shared with the Operations, Finance and Management accounts, and a Finance workbook with the balances, pool, margins and reconciliation figures shared with Finance and Management only; both to named accounts, never by link, through a service account (Roles & Visibility rule 12)). The design answers the review finding on jobs running on two tasks: a lock alone prevents overlap but does not guarantee that a job whose task dies halfway is ever finished.

Rule	What it means
A persisted schedule	One row per job in the database holds its interval, its next due time, the lease and its expiry, and the time of its last start, last success and last failure, updated in place. Nothing about a job lives only in a task's memory, so a restart or a crash forgets nothing. A run row is written only for the daily and hourly jobs, for every failure with its reason, for every lease taken over and for every catch-up after a gap; the frequent jobs leave their own evidence (a market rate row, an expired quote) and no run row. Run history is kept 90 days and failures a year, then pruned: these are operational tables, outside the insert-only rule of Section 3.
One run at a time per job, by a lease that expires	Each job takes its lease before it runs, with an expiry a few seconds ahead that the running task renews while it works, and releases it at the end (Lifecycle & Approvals 6a). A task that finds a live lease exits at once. A task that finds an expired lease takes the job over, because its holder has stopped responding; the take-over is recorded and happens within a minute of the holder dying. Two tasks in two zones therefore never run the same job together, and a job whose task dies is not left for tomorrow.
Checkpoints, so a take-over is safe	Every job works in units (one pair, one deal, one partner, one client journal) and records each unit done in a small in-flight table that is cleared when the run completes. A task that takes a job over, or a retry, starts at the first unit not done, so nothing is done twice. This is safe because no job moves money (Ledger Posting Design, Pattern P): a job stamps a clock, writes a rate, attaches evidence or raises an alert, each once per unit.
A missed-run sweep	Every minute a sweeper reads the schedule and starts any job whose due time has passed and that no live lease holds. A run missed during a deployment, a zone failure or a dead task is therefore started within a minute of the application being back, without anyone noticing it first.
Catch-up after a restart	A job resumes from the last recorded run, not from now: rate capture fills the gap it can, quote clocks expire what passed, the aging job rebuilds the list. A deployment or a zone failure delays a job; it never loses one. The sweep and the checkpoints are what make this true: the sweep starts the overdue run, and the run resumes at its first unit not done.
Bounded retries	A run that fails is retried three times with a growing wait, then left for the next schedule; the failure is recorded with its reason and raises the account alarm after the second failure (Section 6, Monitoring). Nothing is retried forever, and nothing that failed is silently skipped. A take-over from a dead task counts as one of the three attempts, so a job that keeps killing its task stops after three take-overs and alarms instead of passing between the two tasks forever; a stopped job is re-run by Operations from the exception list once the cause is cleared.
A job-health line on the exception list	The exception list every role sees shows, for each job, its last successful run, its last failure and whether it is overdue, so a stale rate feed or a stopped journal writer is visible to Operations before a deal is affected. The line also shows a take-over from a dead task and a job stopped after three attempts, with the button that re-runs it.

None of this moves money: a job still never posts (Ledger Posting Design, Pattern P). Built in Phase 1 with the jobs, on a proven PostgreSQL job library (pg-boss) that provides the schedule, the lease, the retries and the pruning on the database the application already has; no new service; the concurrency test of the go-live acceptance starts a job twice and expects one run (Cutover Approach, exit criterion 10), and its second case kills a task halfway through the daily rebate job, starts the second task and expects exactly one completed result with every unit done once (Test Strategy C-JOBS). An alarm looks like this, by email to Finance, with the job, the time and the event and never a deal figure:

```
Subject: NPLify alarm · daily rebate accrual · stopped after 3 attempts
Job: Daily rebate accrual
Event: stopped after 3 failed attempts
```

```
When: 28 Oct 2026 02:14 HKT (last attempt)
Reason: database connection refused
Units done: 2 of 5 partners (progress kept; a re-run resumes at partner 3)
Next step: Operations re-runs the job from the exception list once the cause is cleared.
```

5 · Screens, API and the visibility rule

There is one application with three views of it, by role. The **Operations** screens are the working day: inquiries and quotes, the quote package, wallet screening, collections to verify, conversions to instruct, settlements to prepare, destination approvals, loss events to record, the client journal; the **Finance** screens add the approvals queue, the partner rate versions and the day's comparison, variance and other exceptions, reconciliation, earnings resting at partners and offsets, invoices and the reseller books, statements, the Finance Log; the **Management** screens add the escalations queue, configuration proposals to approve, system settings, users and roles, and the reports in the reporting currency. Every screen is served by the same API, and the API applies the rule of the Roles & Visibility Matrix at the point where a response is built: for a role whose `ROLE.sees_economics` is false the fields of the groups *Reference rate and comparison*, *Margin, pool, cost breakdown* and *Ledger and balances* are removed before the data leaves the server, on every read, list, export, document and notification. The browser never receives what the role may not see, so there is nothing to hide in the page.

Three API conventions carry the state machines: every state change is a named action (`POST /deals/{id}/actions/instruct-conversion`), never a field edit; an action that needs approval creates the APPROVAL row and returns the state the record is now in; the approver's call is a second action by a different user, checked against ROLE_RIGHT. Idempotency keys on every money-moving action, so a retried request cannot post twice.

6 · Environments, backups and running cost

Environment	Where	Data
Local	Developers' machines	Test data only; the configuration example of the Configuration Schema (config-example-client1.json) and the worked transactions of the Ledger Posting Design as fixtures.
Staging (temporary)	NPL's AWS account: a second small service and database, same image, own bucket	The parallel run of the Cutover Approach: NPL's real configuration, the backfilled deals and every deal mirrored from the spreadsheets for three to four weeks. Its database becomes production's at cutover; the instance is removed afterwards.
Production	NPL's AWS account	Phase test runs before go-live (NPL's page); reset once, just before cutover, and loaded from the staging database so the live ledger starts from reconciled opening entries and the mirrored days, not from test runs.

NPL's page had two environments and a single reset of production before go-live. The cutover plan (a staging instance for the parallel run, then a copy of its database to production) keeps the single reset and adds the temporary staging instance.

- **Backups.** Daily automated snapshots of the database kept 30 days, with point-in-time restore to any minute in that window (RDS automated backups); S3 versioning for files. A weekly restore test (Section 4, last job). A restore is a Management decision recorded in the audit log, because the ledger after a restore is the ledger as of that minute and the sheets fill the gap.
- **Recovery objectives and resilience (NPL decision, 20 October 2026; the region target added 28 October).** Two failure classes, two targets. **A zone failure** (one data centre of the Hong Kong region): the application is usable again within one hour in operating hours, with at most 15 minutes of committed data lost and near zero for a hardware failure, the evidence bucket included; this is the decision of 20 October, unchanged. **A region failure** (the whole Hong Kong region unavailable): the application is usable again within one hour as well, by NPL's choice of 28 October: a live standby database in a second AWS region, kept in step continuously, is promoted, the application is started there from the same image, and DNS is switched; at most 15 minutes of committed data lost, in practice seconds, since the standby is seconds behind the primary; the figure measured at each rehearsal is reported beside the promise. Six things are part of the design rather than options: a Multi-AZ database (a hot standby in a second availability zone with automatic failover, roughly doubling the database line of the running cost); two application tasks in two zones; continuous replication of the database backups and transaction log to the second region (the log segment forced out every minute), from which the database is restored there to any minute; replication of the evidence bucket to the second region; a live standby database in the second region (a cross-region read replica of the primary, promoted to primary on a region failure; its own line in the running cost); and the environment as code, with the secrets replicated and DNS on a short time-to-live, so the application is started in the second region from the repository alone, in the order the runbook gives. The replicated backups stay beside the live standby because they protect against a different risk: a bad change that the standby would faithfully copy. Engineering restores, Finance validates the financial data after any recovery, Management authorises a production restore and the resumption of processing; the steps are the recovery runbook of the Cutover Approach (8a). The fallback workbook, an export refreshed every five minutes to two Google workbooks (Section 4) with open deals, balances, rates, cutoffs and pending approvals, is the tenth job of Section 4 and is built in Phase 2 (M7); five minutes keeps the window Operations re-enters after a failure short.

- **Monitoring.** The app's own exception list is the first monitor; underneath it, the account's basic alarms: service unhealthy, database storage or connections, a job run that failed twice, a rate feed stale for more than ten minutes, a journal row unconfirmed for more than an hour, a job overdue by twice its interval, a job taken over from a dead task, a job stopped after three attempts, and the second-region replication lagging by more than 15 minutes. Alerts go by email to Finance, to the mailbox NPL configures; Finance passes an infrastructure alarm to New XP under the support arrangement. An alarm names the job or the component, the time and the event, and never carries a deal figure, a balance or a margin (Roles & Visibility rule 12); the sample in Section 4 is the shape.
- **Deployment.** One container image per release, built from the repository, migrations applied on start under the migration role, a release note that names the ERD edition the schema matches (the `version` of `erd.v5.json`, 6.0 today).

Schema change policy. Every change to the database is a numbered, forward-only migration kept in the repository with the code that needs it; there is no manual change to a live schema. Before release, each migration is run against last week's restored snapshot (the weekly restore test of Section 4) and the ledger checks are run on the result, so a migration that would break a balance or an invariant is caught on a copy. Migrations on LEDGER_TRANSACTION, LEDGER_POSTING, LEDGER_ACCOUNT and AUDIT_LOG are additive only: a column may be added, never removed or retyped, and no migration updates or deletes a posting. The release note names the migration numbers it carries and the ERD edition the schema matches, and the first start of the new image applies them under the migration role (Section 6, Deployment).

Running cost, line by line

Recalculated on 21 October and 28 October for the design as decided: two application tasks, a Multi-AZ database, a live standby database in a second region with the backups, the log and the bucket replicated there and a small staging environment kept after go-live. Monthly, in USD, region ap-east-1, at today's list prices; the ranges cover the smallest sensible sizes. NPL pays AWS directly; nothing here is a New XP fee.

Line	Basis	USD a month
Application, two tasks	2 × 0.5 vCPU / 1 GB on App Runner or Fargate, running continuously in two zones; the jobs run inside them	50 to 80
Managed PostgreSQL, Multi-AZ	Smallest burstable class with a standby in a second zone (the standby roughly doubles the database line), 20 GB storage, 30-day automated backups, point-in-time restore	60 to 75
Cross-region replication	Continuous replication of the database backups and transaction log to a second region, 30 days kept there, and replication of the evidence bucket; list prices to be confirmed for the Hong Kong region	8 to 15
Second-region standby database	A read replica of the primary in the second region, smallest burstable class, 20 GB storage, kept in step continuously and promoted on a region failure (NPL's choice of 28 October, so that one hour holds for a region failure too)	35 to 45
Files	S3 bucket with versioning: slips, screenshots, photos, quote packages	2 to 4
Logs and alarms	CloudWatch logs for the application and the database, the basic alarms of Section 6	5 to 10
Secrets	Secrets store for the database password, feed and explorer keys, the Sheets service account	2 to 3

Line	Basis	USD a month
Data transfer	Outbound calls to the four services and the Google Sheet, backups to the second region	3 to 8
Staging after go-live	One small task and a single-AZ database of the same image, for releases and the yearly region-recovery rehearsal; stopped outside working hours	40 to 60
Total		**205 to 300**

Against the proposal’s assumption of USD 100 to 200 a month: the first eight lines without the standby, the second task and the cross-region copy would sit at about USD 85 to 125, inside the proposal; the resilience decided on 20 October for the one-hour recovery target adds about USD 80 to 120, which NPL accepted with that decision; the second-region replication and the live standby there, chosen on 28 October so that one hour holds for a region failure too, add about USD 40 to 55 on top. Rate-feed and explorer calls stay within the free tiers at this volume; Google Sheets is free; the authenticator app is the user's own; no licence is needed anywhere in the stack. The staging instance of the parallel run is a further USD 40 to 60 for the weeks it exists, as before.

6a · Security operations

What is already specified stays: NPLify's own login with an authenticator code, a secrets store, encrypted private storage reached through short-lived links, and database roles that cannot alter the ledger. This section adds how security is run after go-live and how NPL raises an incident or a request.

Area	What is done	Who	When
Patching	Operating-system, runtime and dependency updates are applied monthly, and within five business days for a published critical vulnerability in a component the platform uses; each patch release goes through the pipeline of the proposal (build, QA, staging, UAT, production) like any release	New XP under the support bundle (Schedule D)	Monthly; from the eight-week warranty onwards
Privileged access	An access register lists every person who holds the AWS account, the database owner role, the deployment role, the secrets store or the repository, and why; NPL holds the account root and the owner roles; New XP engineers hold time-bound deployment access; the register is reviewed at the quarterly access review (Operating Procedures A.10)	NPL Management owns the register; New XP keeps it current	Quarterly
Key rotation	Database password, feed and explorer keys, the Sheets service account and the deployment credentials are rotated every 90 days and at once when a holder leaves; rotation is a recorded change with no downtime (the secrets store serves the new value to the next start)	New XP	Every 90 days and on a leaver
Vulnerability assessment	A basic automated assessment of the staging environment before go-live, repeated after every major release; findings rated critical or high are fixed before production	New XP	Before go-live, then per major release
Penetration test	The optional add-on of the proposal: an independent OWASP Top 10 test against staging, about USD 1,500 paid to the vendor, over two to three weeks in parallel with delivery; critical and high findings within scope are remediated under the engagement	NPL decides whether to take it	Phase 3, if taken

Raising an incident or a request

Step	During the build	After go-live (support bundle)
1. NPL raises it	Logs it on the shared delivery board (proposal, Section 5) with what was seen, when, and the deal or screen concerned; anything urgent is also a call to the engineering contact	Logs it on the same board under the support bundle (Schedule D) with its priority; a production incident is also a call to the on-call contact
2. New XP acknowledges and classifies	Within the weekly cycle for requests; the same day for anything that blocks the test run	Within the bundle's priority turnaround: a production incident the same business day, a defect within two business days, a request at the next planning
3. What New XP freezes	Nothing in staging is changed until the incident is understood	For a security incident: the affected credentials are rotated at once, deployments are paused, and no data is deleted; a restore is a Management decision under the recovery runbook (Cutover Approach 8a)

Step	During the build	After go-live (support bundle)
4. Who is called	The engineering contact and NPL's point of contact	The on-call engineer, the second named engineer on the support bundle, NPL Management for anything touching money or client data
5. How NPL is kept informed	On the board and at the weekly demo	A first update within the turnaround, then at least daily until closed, and a written note with cause, fix and prevention within five business days of closing
6. Record	The board item	The board item plus the audit log entries for every action taken on production

7 · Decisions for NPL and assumptions made here

1. **Container service.** App Runner (simplest) or ECS on Fargate (more control); both fit the cost line. Assumed: App Runner.
2. **Wallet screening provider**: which tool, and whether the app calls it or Operations pastes the result. Assumed: pasted result with evidence until a provider is chosen.
3. **The restricted store** for KYC, legal documents and bank letters: the same bucket under a restricted prefix (assumed), or a separate secrets store. Bank account details are no longer files: they are rows of BANK_ACCOUNT_DETAILS, read-logged (decision of 26 October 2026).
4. **Client journal content**: which state changes and which fields each client sees; assumed the sender-facing set of MESSAGE_TEMPLATE quote plus state and timestamps, per client.
5. **Availability,** decided on 20 October 2026 with the one-hour recovery target: a Multi-AZ database (a hot standby in a second availability zone with automatic failover) and two application tasks in two zones, so that one zone's failure neither loses the database nor stops the application. The cost of both is in the table of Section 6. A whole-region outage is inside the same one hour since NPL's decision of 28 October 2026: a live standby database in a second region is promoted and the application started there, under the recovery runbook (Cutover Approach 8a), with the fallback workbook bridging the minutes until then; the backups, the transaction log and the evidence bucket are replicated to that region as well. NPL chose this over a four-hour region target at a smaller cost; the standby's line (about USD 35 to 45 a month) is in the table of Section 6.

This note describes the build; the data model it serves is the ERD & Data Model, and the rules it enforces are those of the Lifecycle & Approvals, Roles & Visibility, Calculation and Configuration documents.